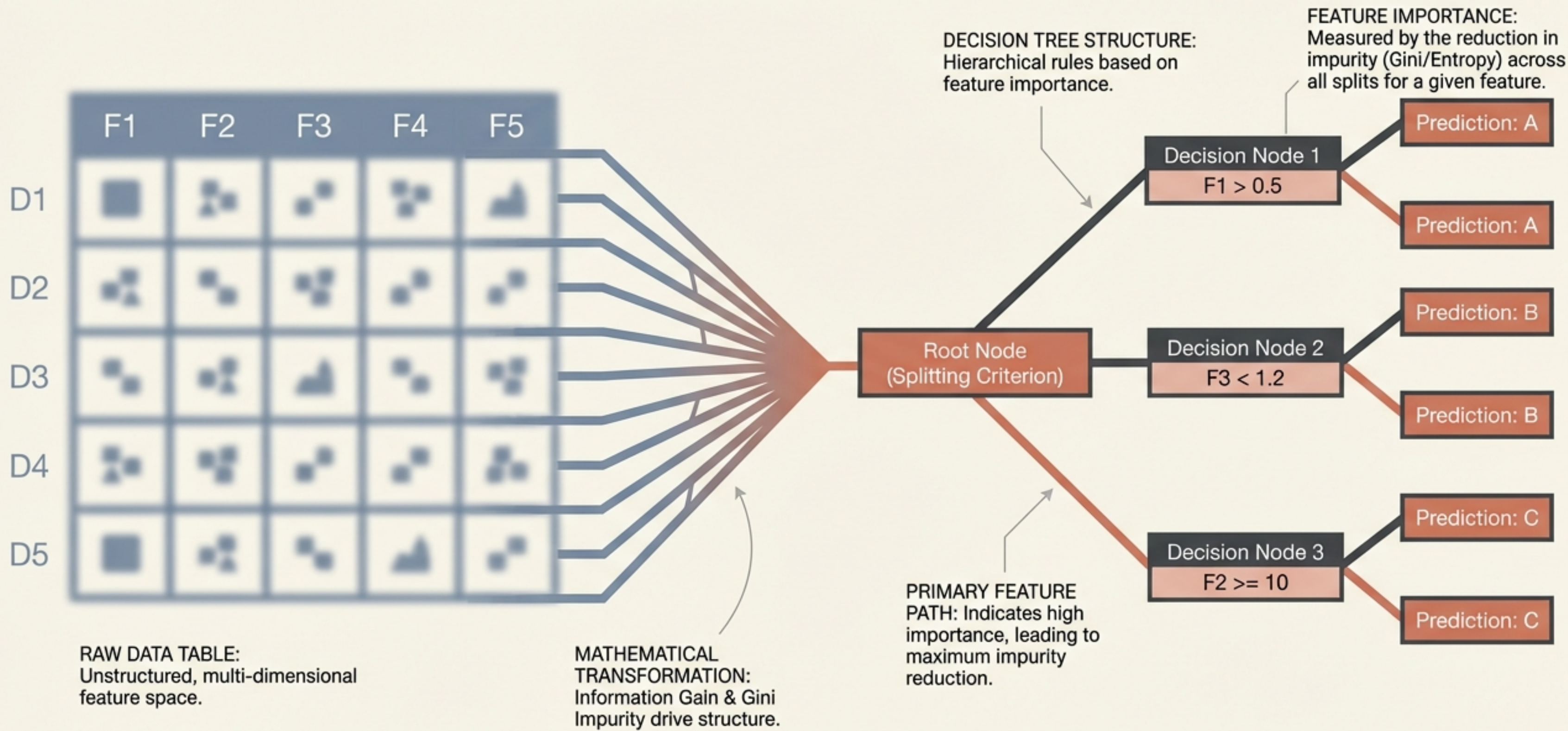


DEMISTIFYING FEATURE IMPORTANCE

Inside the Mathematics of Decision Trees & Random Forests

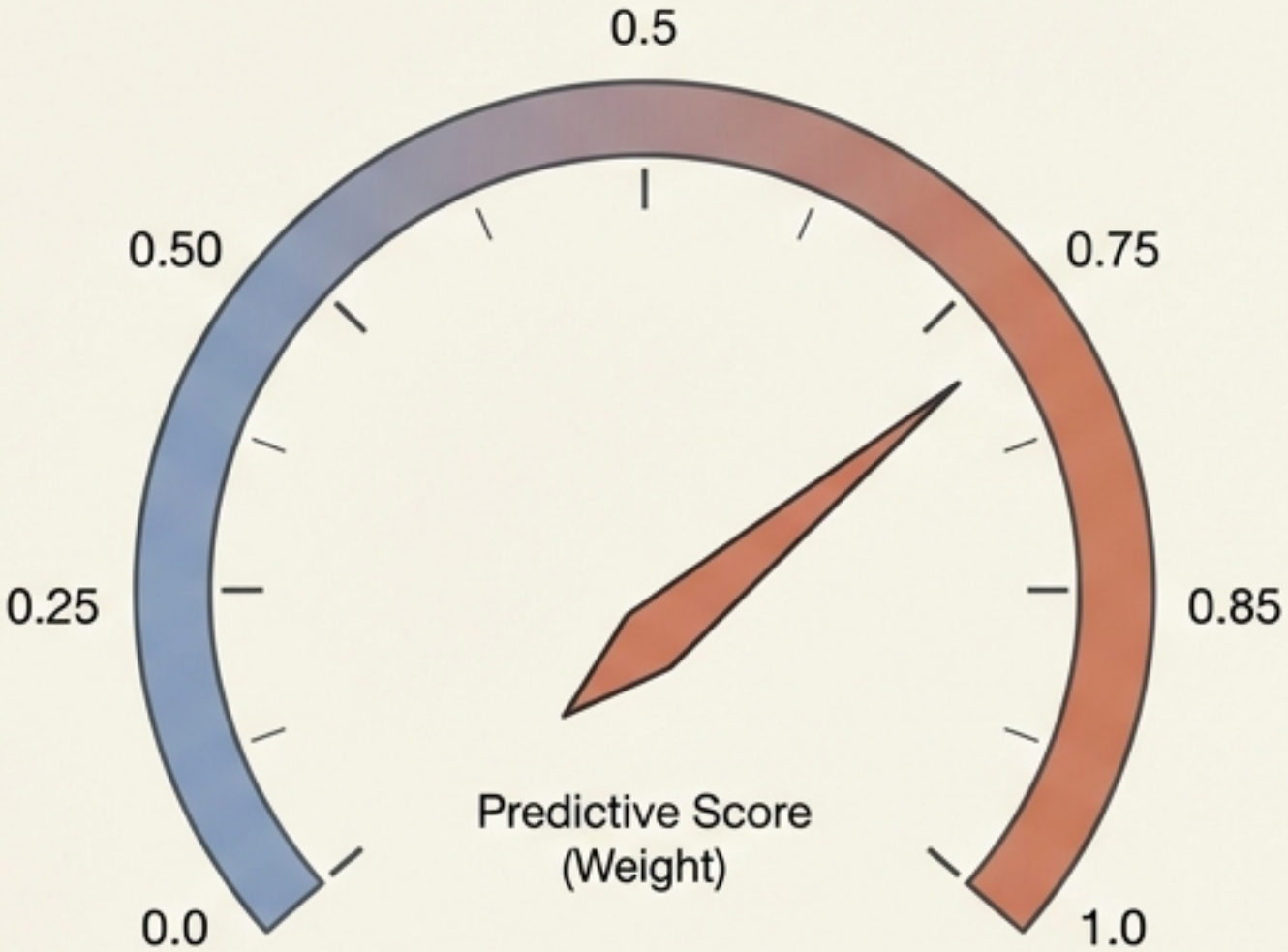


Scoring the predictive power of your data

In machine learning, every column in your dataset is a feature.

Feature Importance is the process of calculating a precise numerical score for each column to determine exactly how useful it is to the algorithm's decision-making process.

		Feature ←		
ID	Date	Feature A	Feature B	Feature C
001	2023-10-27	1.45	10.2	0.89
002	2023-10-28	1.50	10.5	0.92
003	2023-10-29	1.42	10.0	0.87
004	2023-10-30	1.55	10.8	0.95
005	2023-10-31	1.48	10.3	0.90



Why calculate Feature Importance?



1. Feature Selection

Excessive columns slow down training and prevent models from converging. Identifying the best features allows you to drop the useless ones, optimising performance.



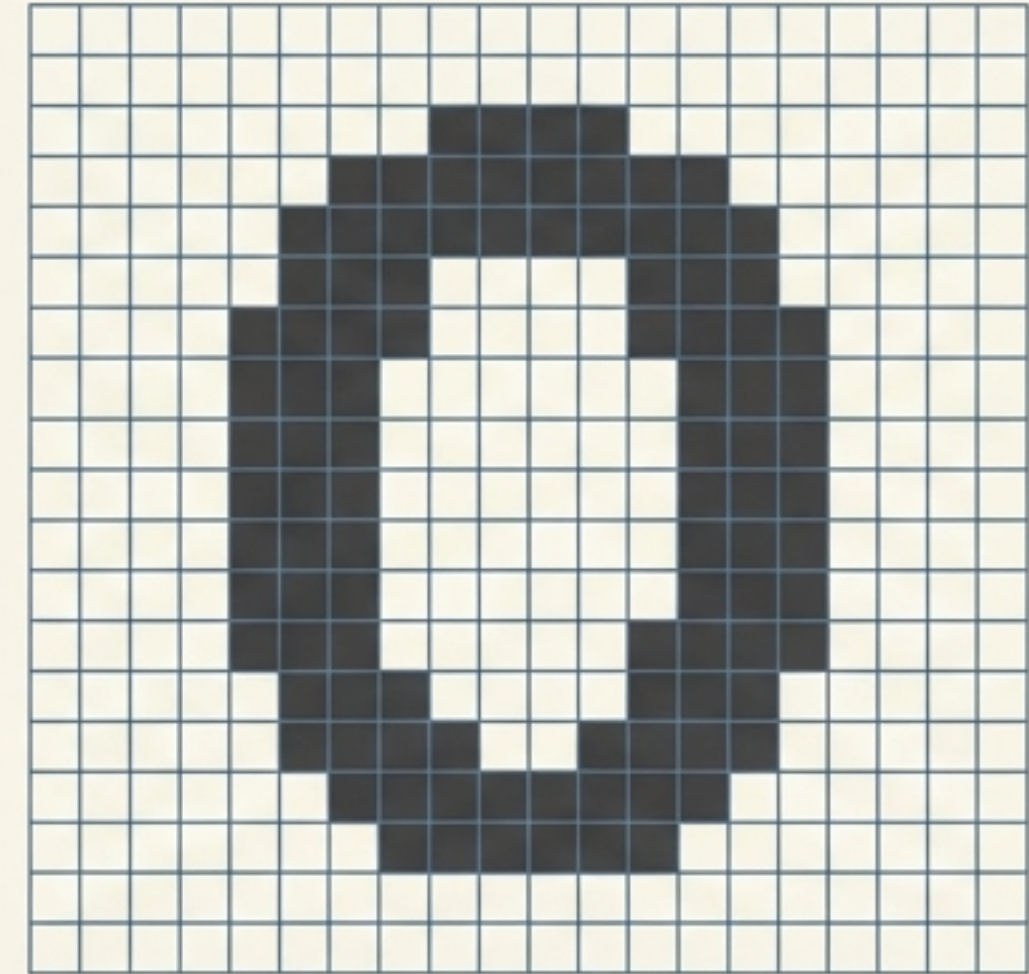
2. Interpretability

Stakeholders demand to know why a model made a decision. If a model rejects a loan application, Feature Importance reveals exactly which variables drove that specific outcome.

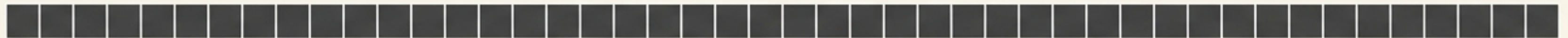
Visualising the problem through MNIST

Consider the famous MNIST dataset of handwritten digits. Each image is a 28x28 pixel grid.

If we flatten this grid, every single pixel becomes a distinct column. That gives our **model 784 individual features** to evaluate.

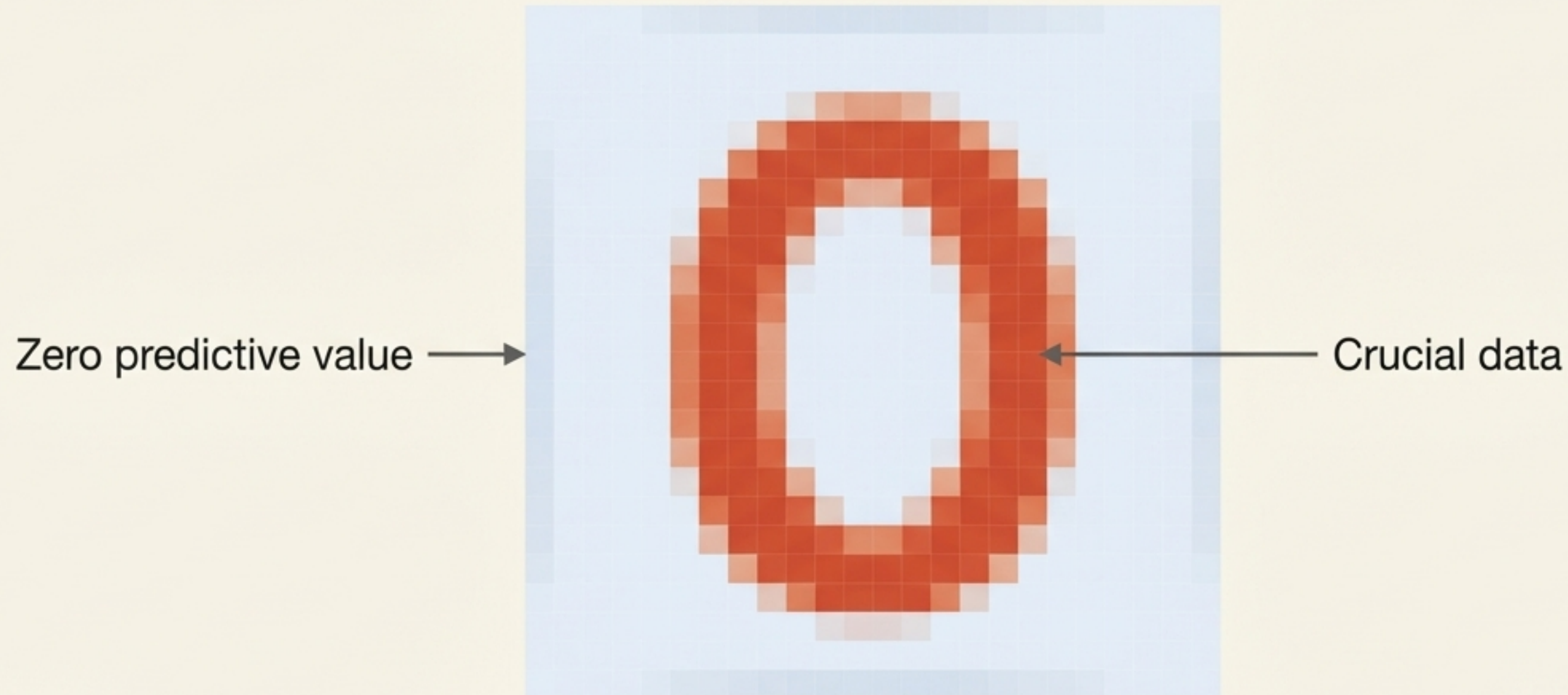


Flattened 1D Feature Vector (784 pixels)



Establishing an intuition for the data

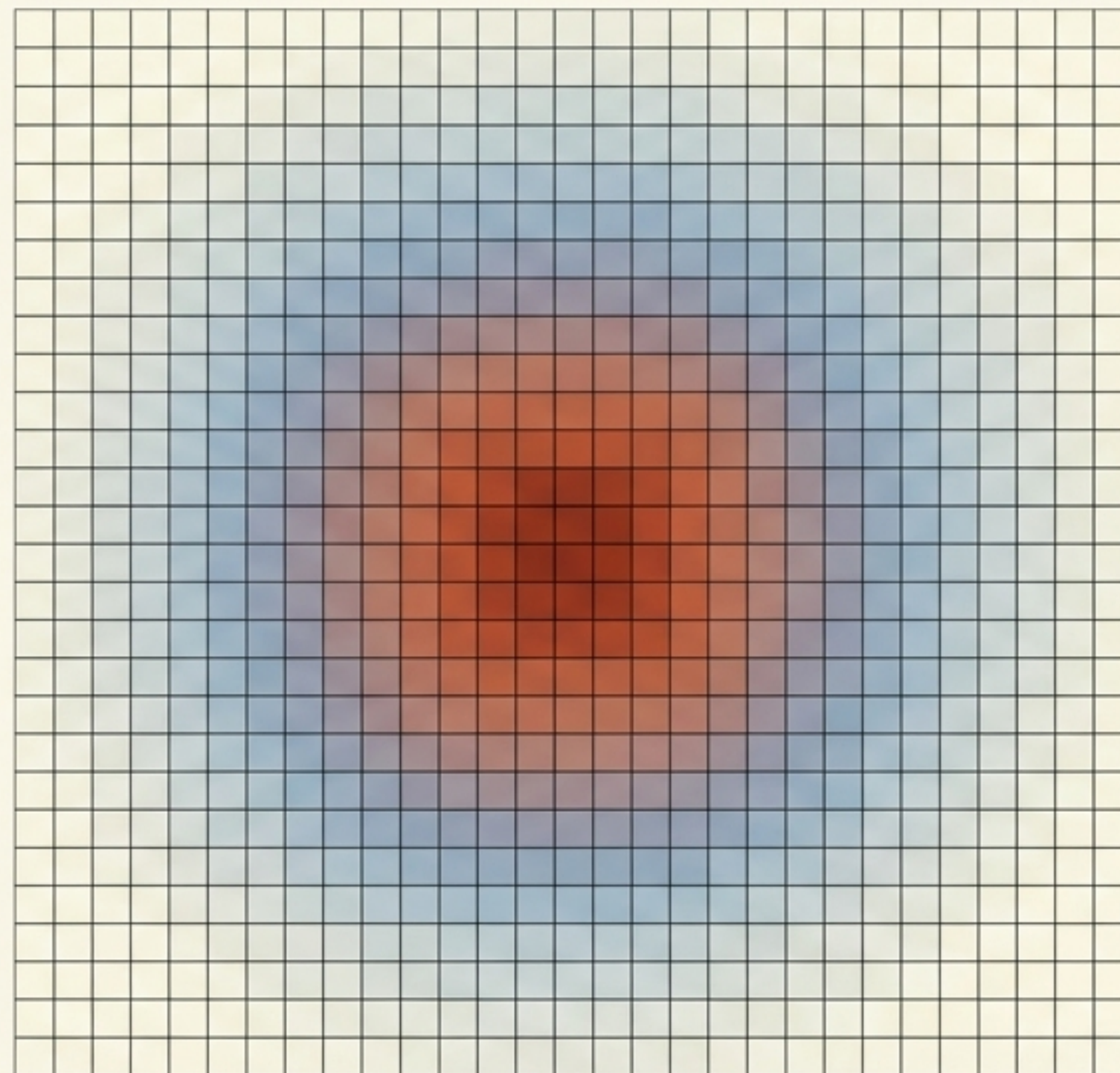
Look at the blank space. Digits are predominantly drawn in the centre of the canvas. The outer edges are almost always empty, regardless of the number. Logically, the centre pixels are highly useful for differentiating a '0' from a '1', while the edge pixels provide zero predictive value.



The Output: Code meets intuition

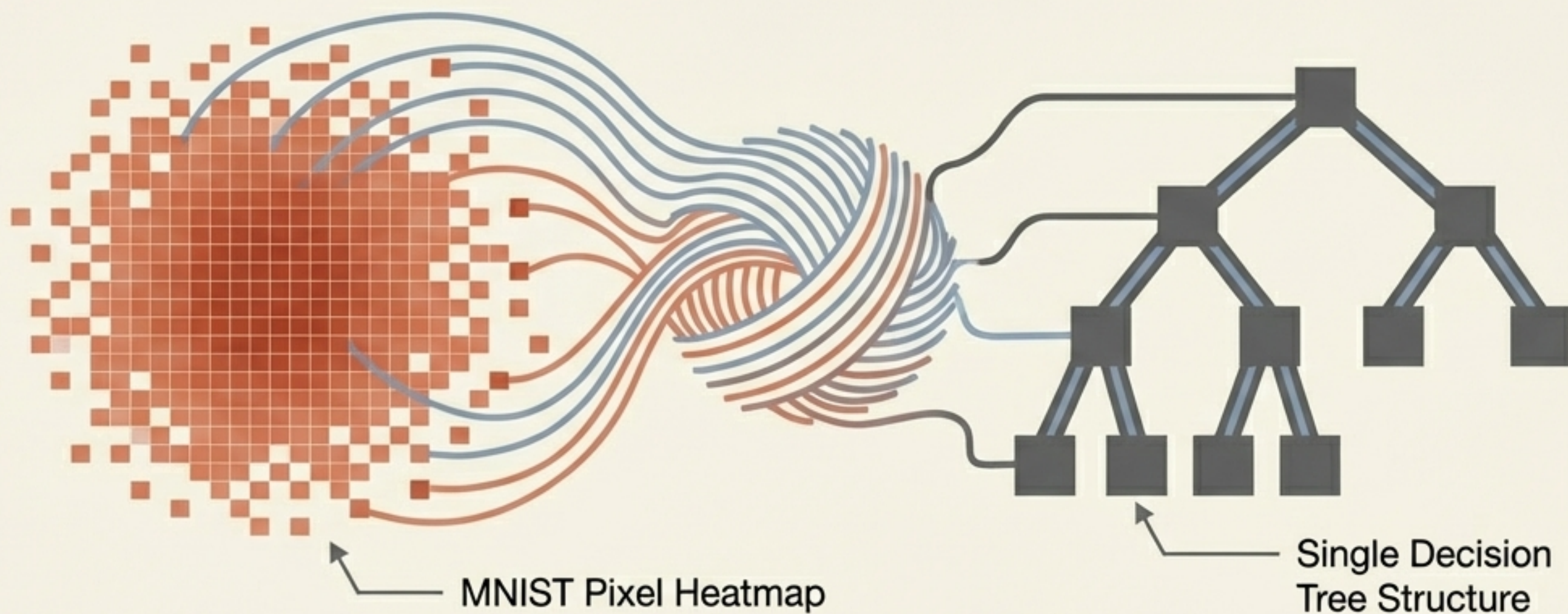
Calling the Random Forest attribute assigns a score to all 784 pixels. When reshaped back into a 28x28 grid, the math perfectly mirrors our intuition: centre pixels show high importance, while the edges fade out.

```
rf.feature_importances_
```



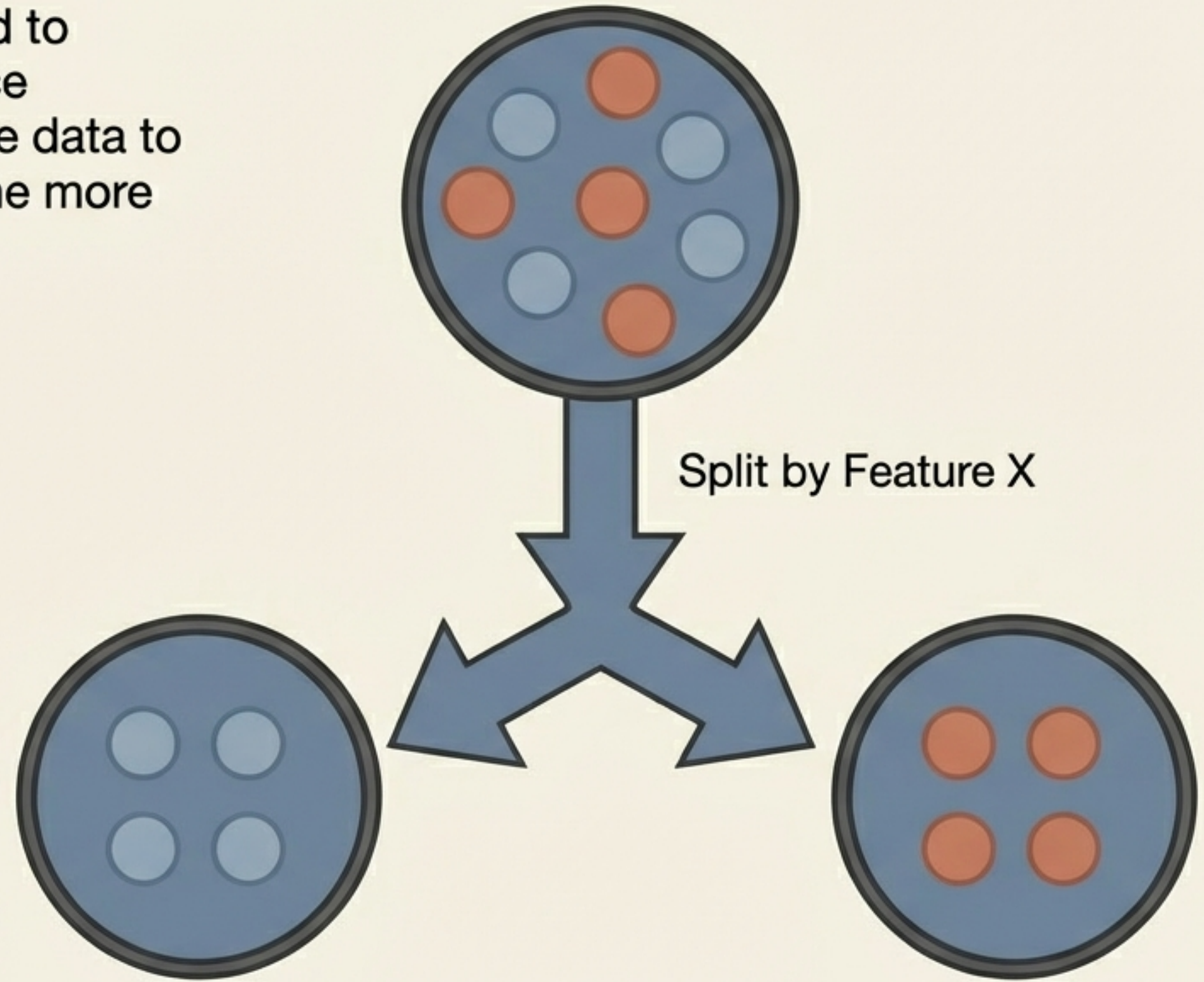
How does the model actually know?

Executing a single line of code is easy, but understanding the underlying engine is what separates practitioners from experts. To understand how a Random Forest calculates importance, we must first dissect a single Decision Tree.



The anatomy of a node split

Inside a Decision Tree, features are used to split the data. A feature gains importance based on how effectively it organises the data to reduce impurity. The cleaner the split, the more important the feature.



The Node Importance Formula

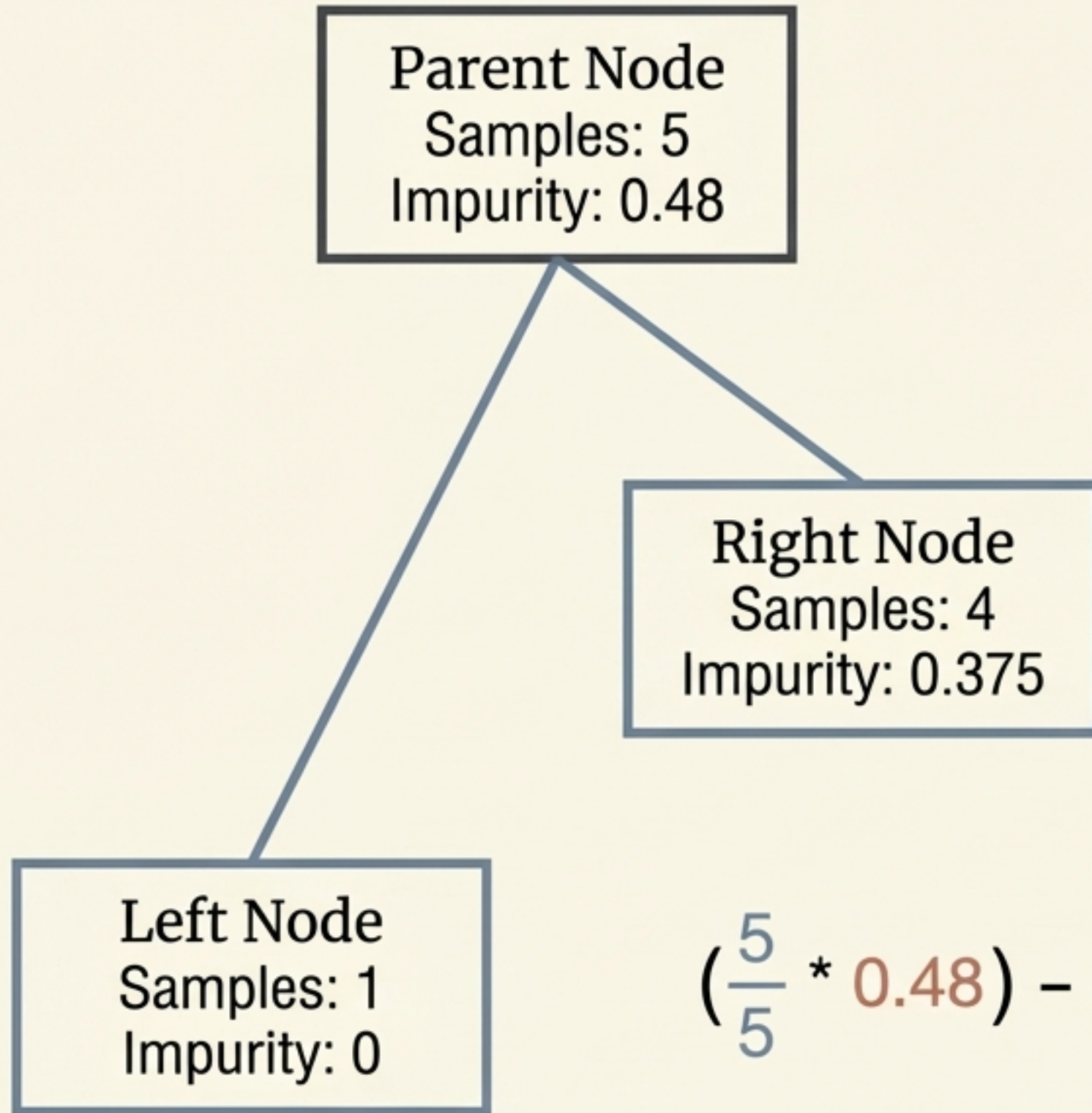
Scikit-learn defines feature importance as the total reduction of a criterion brought by that feature. Mathematically, it calculates the weighted impurity of the parent node and subtracts the weighted impurities of both child nodes.

$$\text{Node Importance} = \frac{N_t}{N} * \text{Impurity} - \left[\frac{N_{t_Right}}{N} * \text{Right_Impurity} \right] - \left[\frac{N_{t_Left}}{N} * \text{Left_Impurity} \right]$$

- **N_t** : Samples at this specific node
- **N** : Total samples in the entire dataset
- **Impurity** : The Gini score of the node

Calculating the reduction in impurity

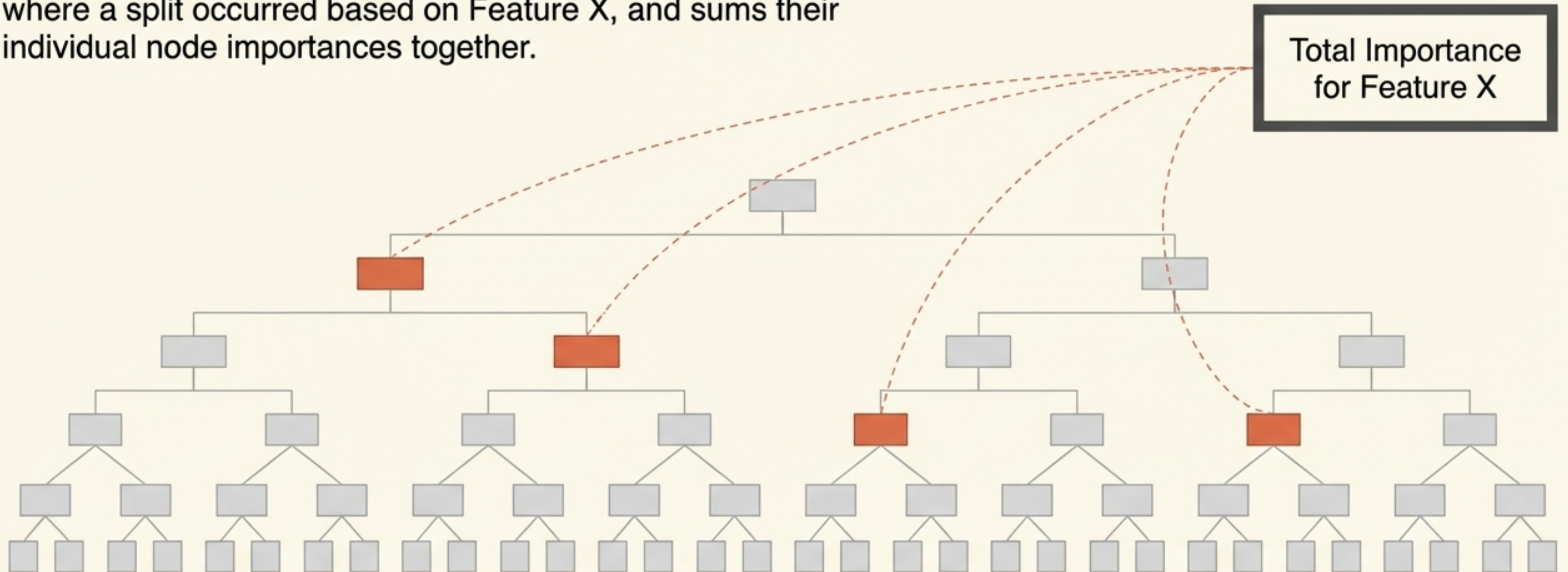
By plugging the sample counts and Gini scores into the formula, we calculate exactly how much a specific node reduced the overall chaos in the dataset. This isolated value is the Node Importance.



$$\left(\frac{5}{5} * 0.48\right) - \left(\frac{4}{5} * 0.375\right) - \left(\frac{1}{5} * 0\right) = \text{Node Importance}$$

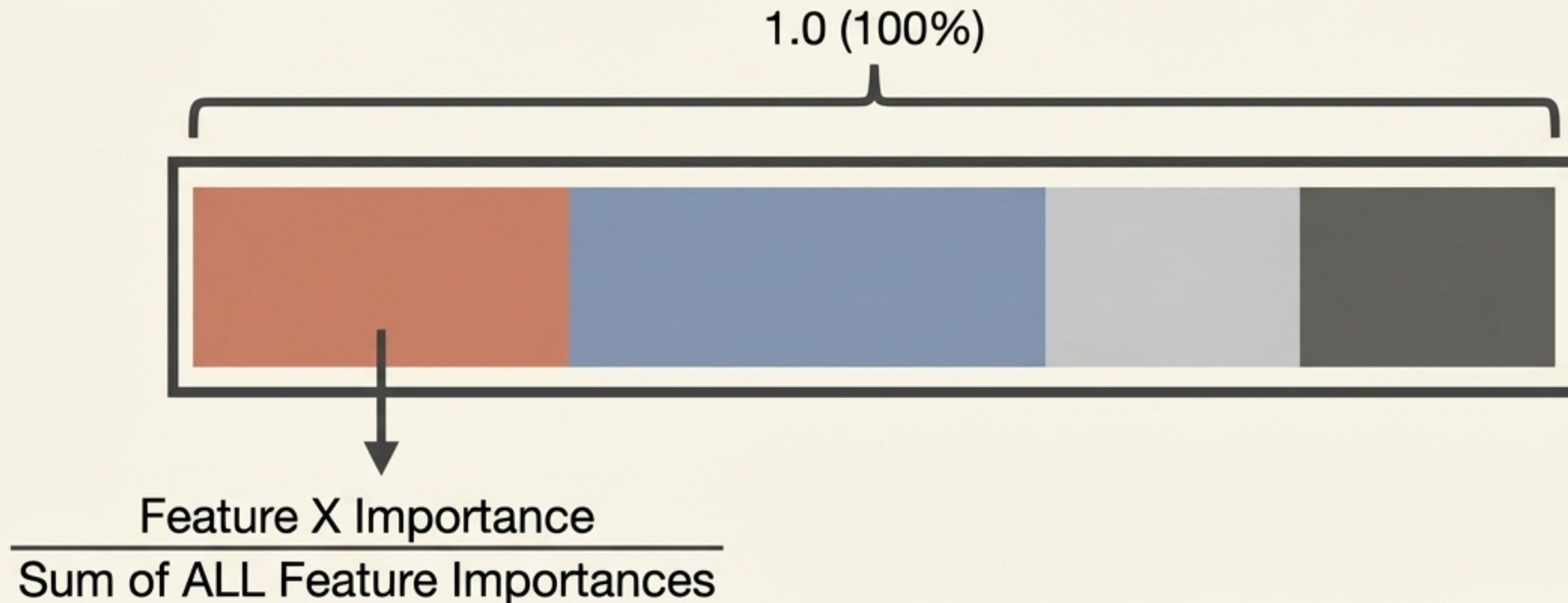
Feature-level aggregation

A single feature might be used to split the data multiple times across different levels of the tree. To find the total importance of Feature X, the algorithm identifies every single node where a split occurred based on Feature X, and sums their individual node importances together.



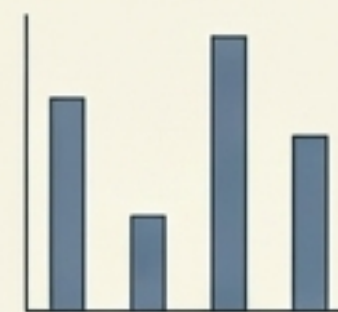
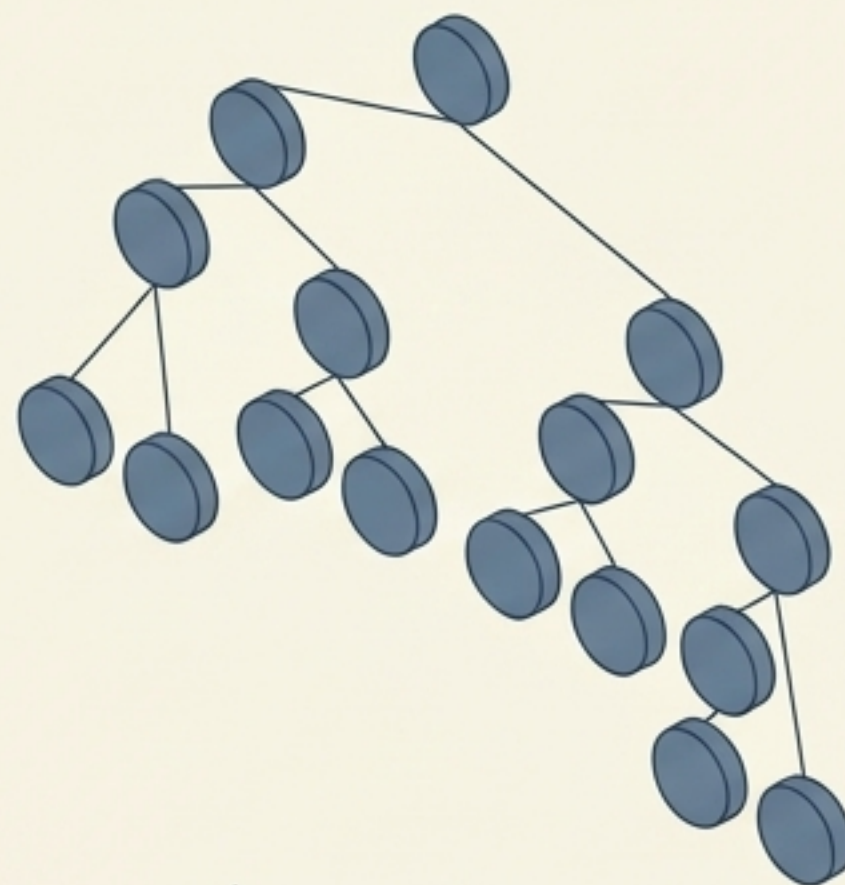
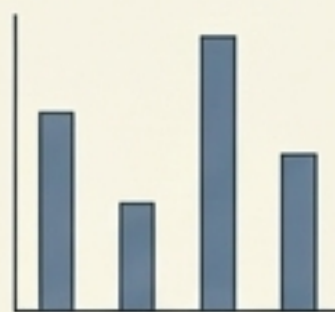
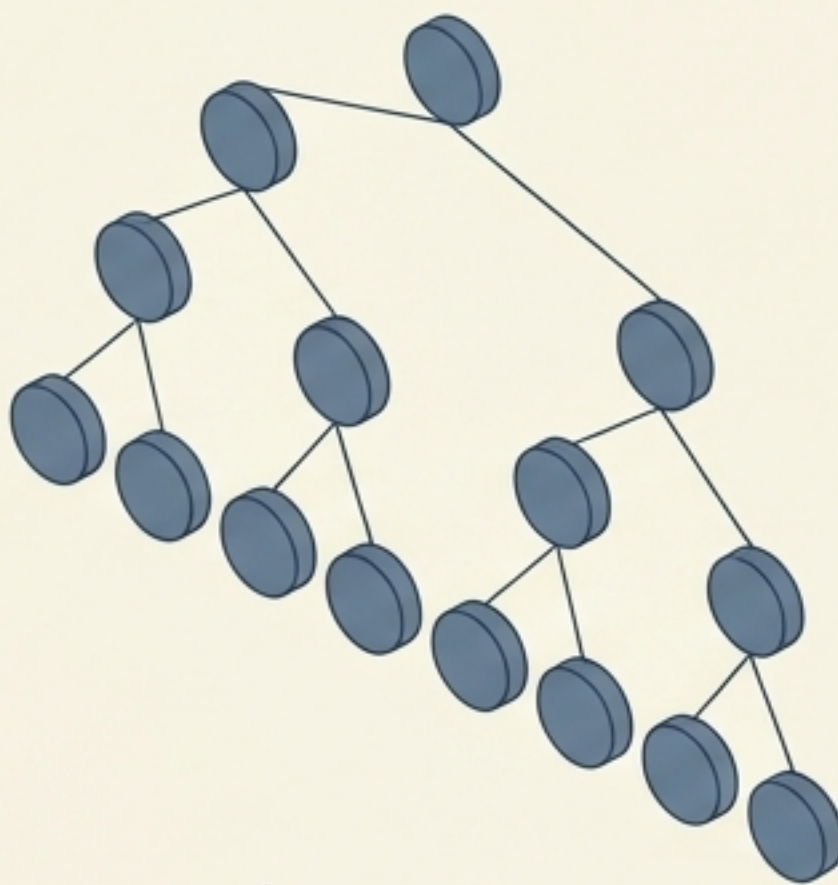
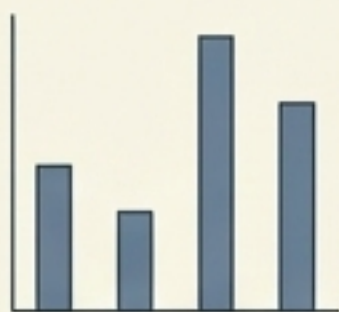
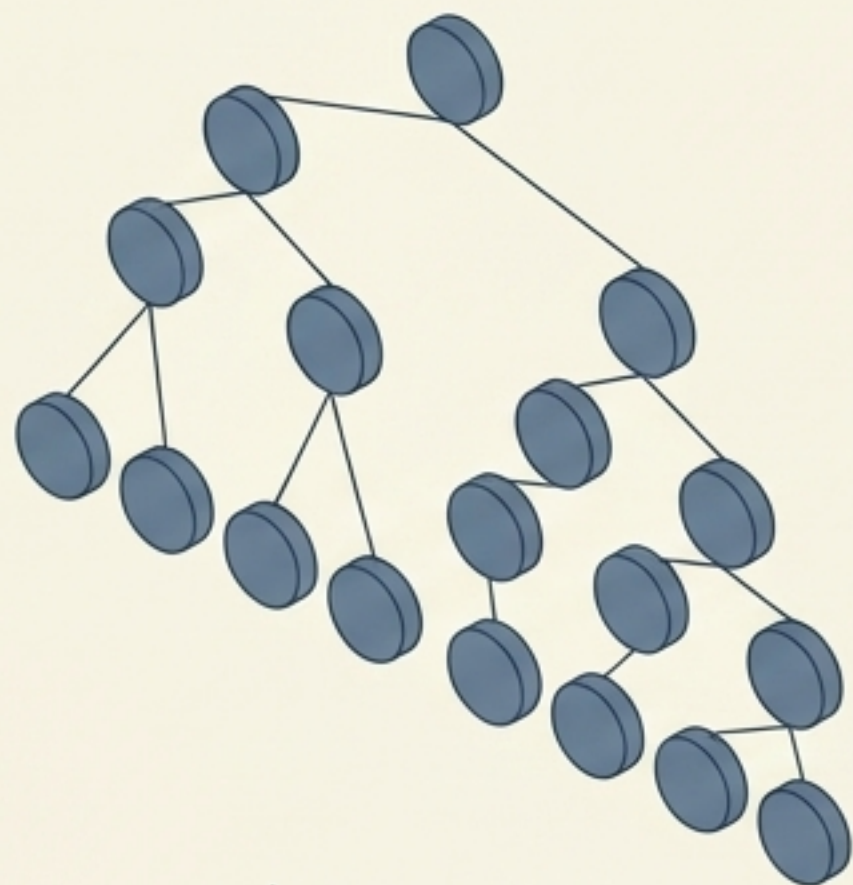
Normalising the final scores

Raw importance scores are difficult to compare. Scikit-learn normalises these values by dividing the individual importance of each feature by the total sum of all feature importances in the tree. This ensures that the final scores always equal exactly 1 (or 100%).



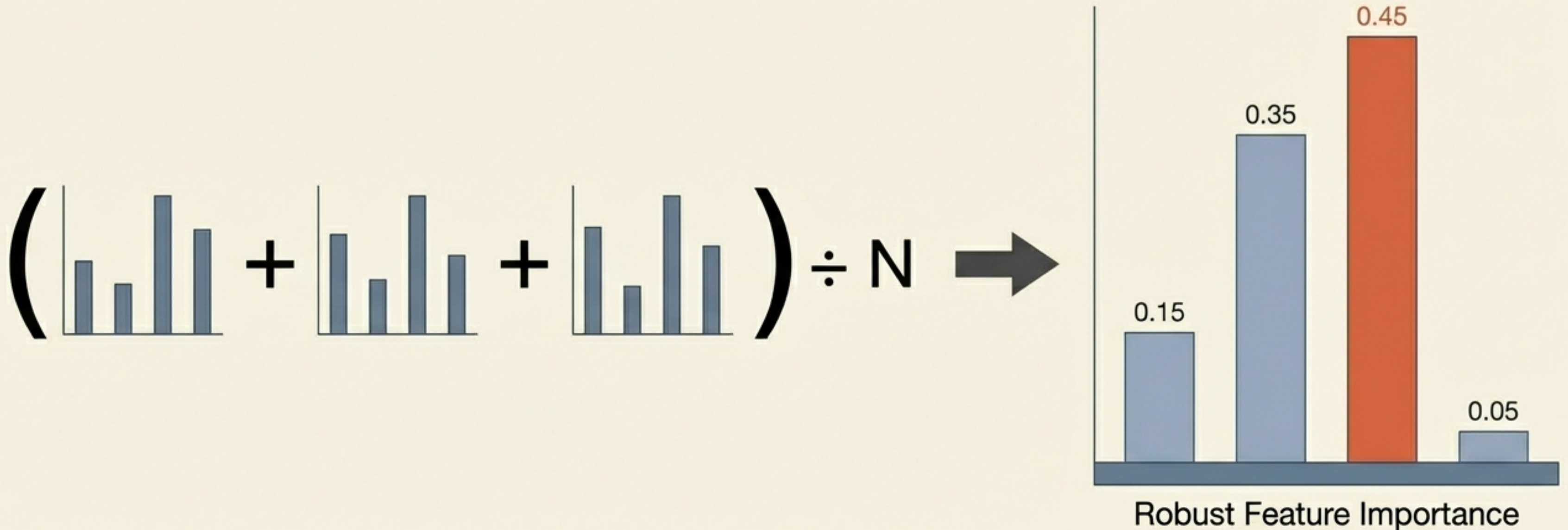
Scaling up to Random Forests

A Random Forest is simply an ensemble of multiple Decision Trees. Because each tree in the forest is trained on slightly different subsets of data, each individual tree will calculate a slightly different feature importance array.



The averaging engine

The Random Forest algorithm aggregates the normalised feature importance scores from every single tree in its ensemble, calculating the mean average for each feature. This averaging process smooths out anomalies and produces a highly robust, reliable evaluation of your data.





Critical Caveat: High Cardinality Features

Impurity-based feature importance has a known mathematical flaw: it can be heavily misleading for high-cardinality features (columns with many unique values, like IDs or continuous variables). The algorithm naturally biases toward them. If your dataset contains **high-cardinality features**, **bypass this method** and utilise scikit-learn's **Permutation Importance** instead.